



SAP Application Development (S/4 HANA)

ABAP Lock Concept

Content

This chapter is about the ABAP lock concept. You will learn how to use a previously defined lock object to ensure data consistency even with multiple concurrent write operations on the same data set.

Prerequisites

You should be familiar with ABAP and principles of ABAP Objects with database access. Besides, it is necessary to have experience SAP GUI and Eclipse navigation. For this, you should have completed the chapter *Introduction to SAP Programming*.

Motivation

Since shared data resources and operational concurrency using multiple applications simultaneously is state-of-the-art today, it is necessary to apply the concept of locks to guarantee data consistency. This chapter is an excellent introduction to learn how to use lock objects for SAP applications.

Lecture notes

Fundamental knowledge about database access for ABAP programs and principles in both ABAP and ABAP Objects development is a prerequisite for the students. Students can go on with their account from the previous chapters.

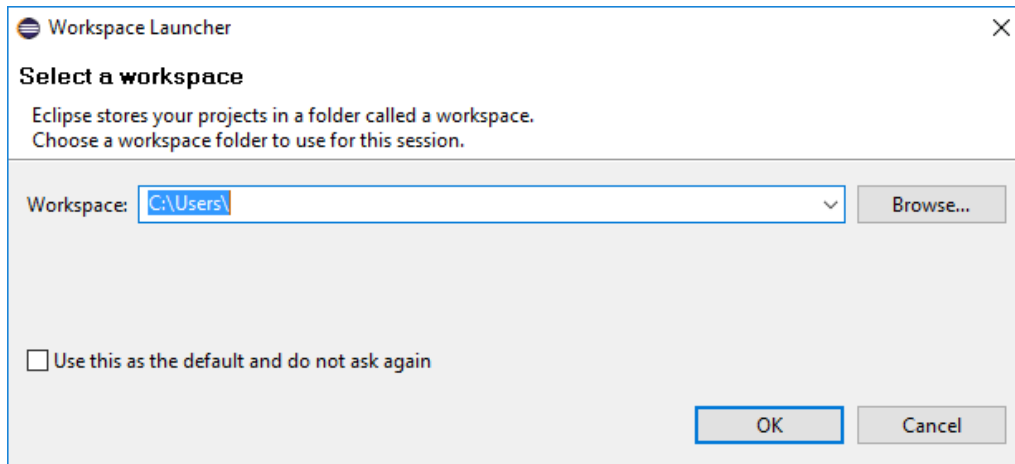
- **Product:** All
- **Level:** Advanced
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Login into the SAP system within Eclipse

Short description: Use Eclipse to login into the SAP system with your username and password

Start Eclipse and select the local workspace where your ABAP project is stored.

Login



Please refer to the local workspace of **Module “Dynamic Programs”** where you learned how to access and work on data sets from the database. If you haven't finished the exercise from **Module “Dynamic Programs”**, please do it now before you continue with this one.

Task 2: Usage of lock object

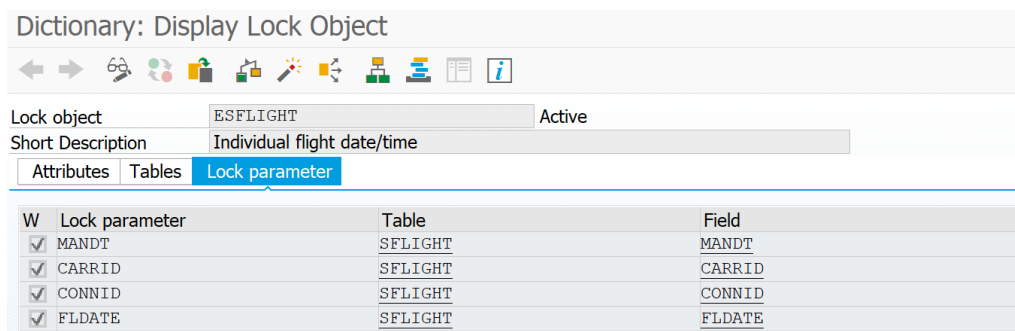
Short description: Use lock objects in your program from **Module “Dynamic Programs”** to avoid data inconsistency when changing connection details

At first you want to have a look on the lock object itself. Please open the following path within the Project Explorer to access the dictionary information of the lock object 'ESFLIGHT':

System Library • S_NWDEMO_BASIS • SAPBC_MAIN • SAPBC_GLOBAL • Package path Dictionary • Lock Objects • ESFLIGHT

The system comes up with the details about the lock objects. Switch to the tab '**Lock parameter**' and have a look on the lock parameter. You see that the lock object uses the primary key of the table '**SFLIGHT**' to lock data sets in the table.

ESFLIGHT



Important Hint:

Please note that customer lock objects are bound to a namespace too. Name all of your custom lock objects to **EZ_<table name>**. You can create a new lock object after you created the table to which it belongs.

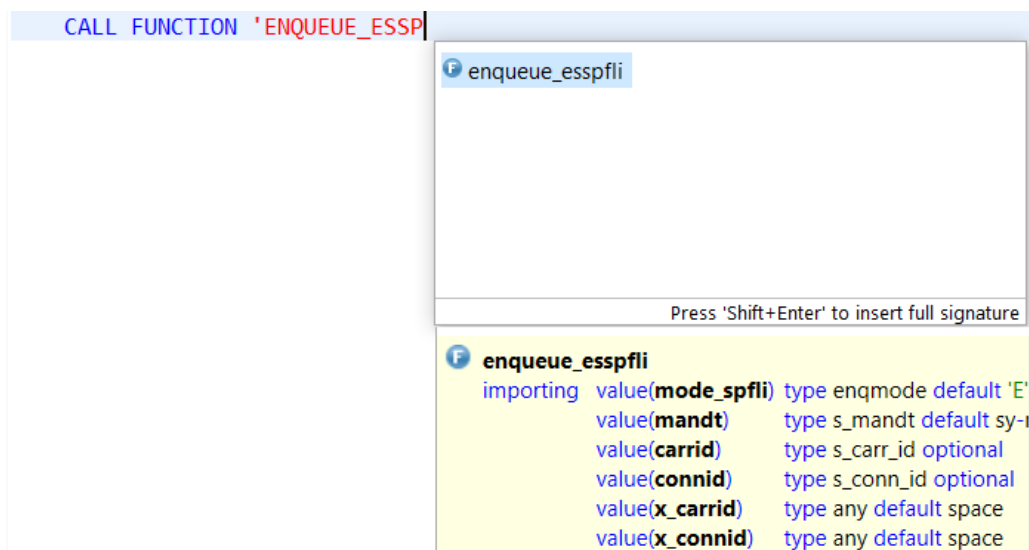
**Important
hint**

Now you want to use a lock object in your program from the previous module. The ideal way to use it is to implement two locks: one shared lock when reading the data from the table 'SPFLI' and one exclusive lock when writing the data to table 'SPFLI'. You start with the shared lock in 'USER_COMMAND_0100'. Please refer to the previous program called **Z \$\$ _DYNPRO** and open the module. The module is listed in the folder 'PAI Modules'. Add the shared lock just in front of the 'SELECT' statement - to do so place the cursor before the 'SELECT' statement, type in **CALL FUNCTION 'ENQUEUE_ESSP** and press **Ctrl + Space** on your keyboard to call the auto completion popup.

**Call
Function**

Ctrl + Space

Shift + Enter



Select **enqueue_esspfi** and press **Shift + Enter** as proposed in one of the popup windows on your keyboard to retrieve all necessary parameters from the system automatically.

```
CALL FUNCTION 'ENQUEUE_ESSPFI'
*   EXPORTING
*       mode_spfli      = 'E'          " Lock mode for table SPFLI
*       mandt           = SY-MANDT     " 01th enqueue argument
*       carrid          =              " 02th enqueue argument
*       connid          =              " 03th enqueue argument
*       x_carrid        = SPACE        " Fill argument 02 with initial value?
*       x_connid        = SPACE        " Fill argument 03 with initial value?
*       _scope          = '2'
*       _wait           = SPACE
*       _collect        = ' '          " Initially only collect lock
*   EXCEPTIONS
*       foreign_lock    = 1
*       system_failure  = 2
*       others          = 3
* .
IF sy-subrc <> 0.
*   MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
*           WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
ENDIF.
```

To use it properly you have to modify the function call at first. Please use the following example:

Source code

```
MODULE user_command_0100 INPUT.
```

```

CASE ok_code.
  WHEN 'BACK'.
    LEAVE PROGRAM.
  WHEN 'SELECT'.
    CALL FUNCTION 'ENQUEUE_ESSPFLI'
      EXPORTING
        mode_spfli      = 'S'
        mandt           = sy-mandt
        carrid          = spfli-carrid
        connid          = spfli-connid
      EXCEPTIONS
        foreign_lock    = 1
        system_failure = 2
        others          = 3.

    SELECT SINGLE * FROM spfli INTO wa_flight
      WHERE carrid = spfli-carrid AND connid = spfli-connid.

ENDCASE.

CLEAR ok_code.

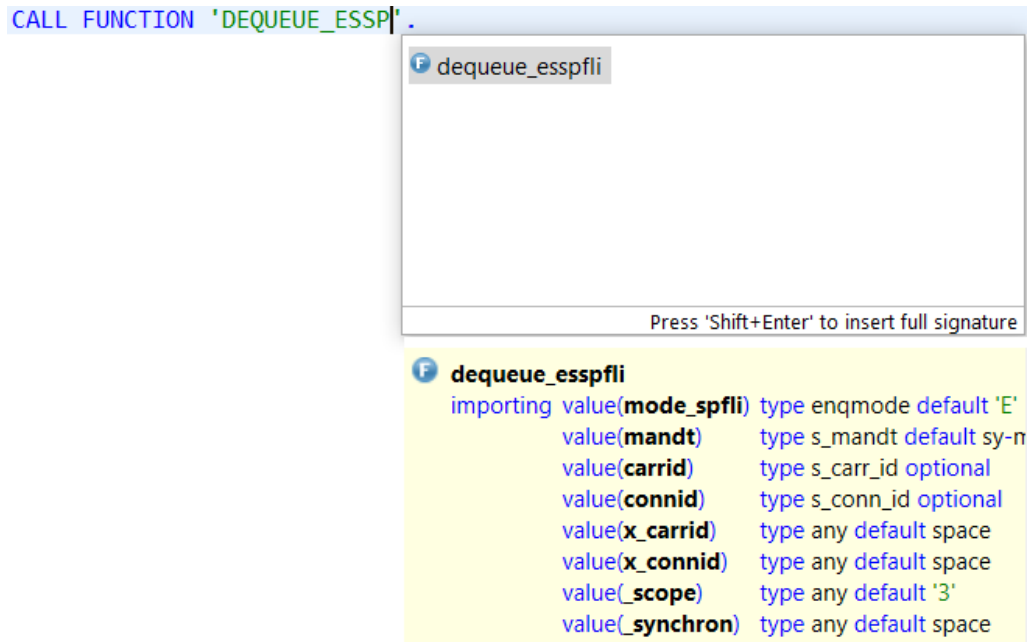
ENDMODULE.
```

To call the enqueue function you have to use the current selected '**CARRID**' and '**CONNID**' from your structure '**SPFLI**'. Moreover you have to specify the type of lock ('**S**' for shared lock) and the current client (this is read from system variable '**SY-MANDT**'). Save, check and activate your module.

*Save, check,
activate*

This shared lock will be released right after the user clicks a button in the second screen of your program. Hence, please open your '**USER_COMMAND_0200**' module. The shared lock will be released shortly before the case instruction. Place the cursor before the case instruction, type in **CALL FUNCTION 'DEQUEUE_ESSP** and press **Control + Space** on your keyboard to call the auto completion popup.

**USER_COM
MAND_0200**



The parameters for the function call are very important as you have to specify the lock type 'S', the 'CARRID' and 'CONNID'. Make sure you entered the correct parameters:

```

CALL FUNCTION 'DEQUEUE_ESSPFI'
  EXPORTING
    mode_spfli = 'S'
    mandt      = sy-mandt
    carrid     = spfli-carrid
    connid     = spfli-connid.

```

Unlock

Now as you have unlocked the data set you are allowed to set an exclusive lock for changing the data set. Once again you use auto-completion function and insert the function call to create an exclusive lock. Right after the data modification you release the lock again. Your source code should look similar to this:

Source code

```

MODULE user_command_0200 INPUT.

* Release shared lock
CALL FUNCTION 'DEQUEUE_ESSPFLI'
  EXPORTING
    mode_spfli = 'S'
    mandt      = sy-mandt
    carrid     = spfli-carrid
    connid     = spfli-connid.

CASE ok_code.
  WHEN 'SAVE'.
    IF spfli <> wa_flight.

* Create exclusive lock
CALL FUNCTION 'ENQUEUE_ESSPFLI'
  EXPORTING
    mode_spfli = 'E'
    mandt      = sy-mandt
    carrid     = spfli-carrid
    connid     = spfli-connid
  EXCEPTIONS
    foreign_lock = 1
    system_failure = 2
    others       = 3.

* Now modify data set
MODIFY spfli FROM spfli.

* Release exclusive lock
CALL FUNCTION 'DEQUEUE_ESSPFLI'
  EXPORTING
    mode_spfli = 'E'
    mandt      = sy-mandt
    carrid     = spfli-carrid
    connid     = spfli-connid.
ENDIF.
ENDCASE.

CLEAR ok_code.
ENDMODULE. "USER_COMMAND_200 INPUT

```

Please make sure you do not forget the point after each function call! Save, check and activate your program.

**Save, check,
activate**

When testing your new program, you may monitor the creation of the shared lock in transaction **SM12**. When starting the transaction please enter the name of the table **'SPFLI'** and hit the button **'List'**. You see a list of all your current existing locks.

Connection details

Lock Entry List Second Window (use SM12)

Refresh Details [Icons]

Client	User Name	Date/Tim	Lock Mod	Table name	Argument	Count	Dial Counter	Update
200	DEV-210	09:28:13	S	SPFLI	200AA 0017	0		1

Client: 200
 Airline: AA
 Flight Number: 17
 Country: US
 Depart.city: NEW YORK
 Dep. airport: JFK
 Country: US
 Arrival city: SAN FRANCISCO
 Dest. airport: SFO
 Flight time: 6:01
 Departure: 11:00:00
 Arrival Time: 14:01:00
 Distance: 2.572
 Distance in: MI
☐ Charter
 n day(s) later:

Challenge:

Short description: Extend your program in order to see an acquired exclusive lock when you call your program and open the screen to change the flight dates. Differentiate between view and change mode. Calling transaction *SM12* within the SAP GUI will give you the opportunity to monitor current locks.